

Multi-Agent AI Code Review Platform

Architecture, data flow, and project structure

An event-driven, multi-agent system that reviews GitHub pull requests using specialized Spring AI agents coordinated through Kafka event choreography and a Redis blackboard, grounded in a RAG-based index of the codebase.

1. Project Overview

This system reviews pull requests the way a team of specialized reviewers would: in parallel, not sequentially. Instead of one large prompt asking an LLM to "review this diff," the PR is handed to several independent agents — static analysis, security, style, test coverage, and documentation — each with its own narrow tool set and its own reasoning pass. They never call each other directly; they communicate by writing to a shared Redis blackboard, and an aggregator agent reconciles their findings into a single, coherent review comment.

The system is grounded with retrieval: a background indexing pipeline continuously parses the repository with tree-sitter, embeds each function/method as a vector, and stores it in Postgres via the pgvector extension. When a PR is reviewed, a context builder agent retrieves the most semantically similar existing code so that agents can compare a new change against the codebase's actual conventions, instead of judging it in isolation.

One-line resume summary

Multi-agent AI code review system — Spring AI agents (static analysis, security, style) coordinated via Kafka event choreography and a Redis blackboard, with RAG-grounded context retrieval, producing a single synthesized PR review.

Core architectural patterns demonstrated

- **Orchestrator-Worker:** a central orchestrator decomposes each PR into subtasks and dispatches them to independent worker agents.
- **Blackboard pattern:** agents share state through Redis rather than calling one another directly — each reads context, writes findings, and remains unaware of the others.
- **Event choreography over orchestration-by-call:** services communicate through Kafka topics, which decouples them in time and makes each one independently scalable and replaceable.
- **Retrieval-Augmented Generation (RAG):** agents reason over the diff plus a handful of retrieved, semantically similar code chunks, not the whole repository and not a bare diff.

2. Full Architecture

The diagram below shows the complete system: a background indexing branch on the left that runs continuously, and the main pull-request review pipeline running top to bottom.

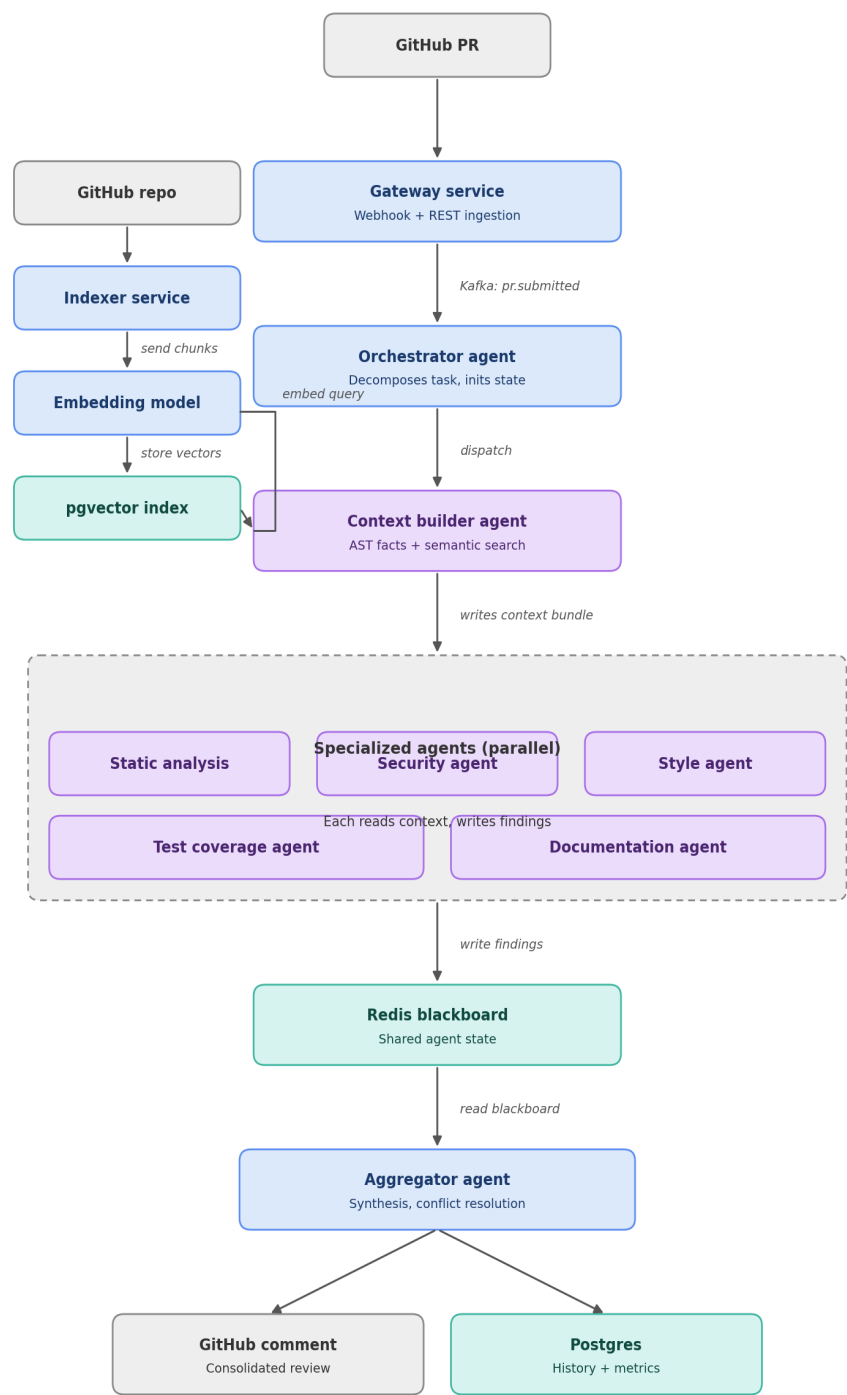


Figure 1 — complete architecture: background indexing (left) and the PR review pipeline (center).

2.1 Background indexing (runs continuously, independent of any PR)

- **GitHub repo** — the source of truth. A push to `main` triggers re-indexing; this is a separate trigger from a PR being opened.
- **Indexer service** — parses changed files with tree-sitter, extracts symbol-level chunks (whole functions/methods, not fixed-size text windows), and computes a content hash per chunk so unchanged code is never re-embedded.
- **Embedding model** — a shared local service (e.g. a sentence-transformers sidecar) that turns chunk text into vectors. It is called from two places: by the Indexer at index time, and by the Context builder at query time — using the same model both times is required, since embeddings from different models aren't comparable.
- **pgvector index** — Postgres with the pgvector extension, storing each chunk's text, metadata, and vector, searchable by cosine distance.

2.2 Request intake

- **GitHub PR** — the trigger: a PR opens or receives a new commit.
- **Gateway service** — pure infrastructure, no LLM involved. Verifies the GitHub webhook signature (HMAC), parses the diff payload, and publishes a `pr.submitted` Kafka event.
- **Orchestrator agent** — decides which agents actually need to run for this PR (skip the security agent on a docs-only change) and initializes the PR's entry on the Redis blackboard.
- **Context builder agent** — combines two kinds of context: deterministic AST facts parsed fresh from the diff (exact call graph, signatures) and semantic retrieval from the pgvector index (similar existing code). Packages both into a context bundle and writes it to the blackboard before any specialized agent runs.

2.3 Specialized agents (parallel tier)

All agents read the same context bundle from the blackboard, run independently and concurrently, and write their own findings back — this is the blackboard pattern in practice.

- **Static analysis agent** — wraps a real linter (PMD/Checkstyle/ESLint) and uses the LLM to filter false positives and explain root cause.
- **Security agent** — checks dependencies for known CVEs and uses the LLM to reason about logic-level risks static tools miss.
- **Style agent** — retrieves the team's actual coding guidelines via RAG so feedback matches real conventions, not generic best practice.
- **Test coverage agent** — deterministically finds coverage gaps, then drafts tests for them.
- **Documentation agent** — retrieves existing doc style, then generates documentation that matches it.

2.4 Synthesis

- **Redis blackboard** — shared, short-lived state per PR: the context bundle going in, every agent's findings coming out, keyed by PR id with a TTL since it's working memory, not permanent storage.

- **Aggregator agent** — waits for all dispatched agents to report (with a timeout so one slow agent can't block the review), resolves conflicting findings, and produces one consolidated review.
- **GitHub comment** — the synthesized review, posted back via the GitHub REST API.
- **Postgres** — persists the full review record for history and later benchmarking (precision/recall against a plain linter).

3. How Background Indexing Works, Step by Step

The indexing service's entire job is to make retrieval possible: turning source code into small, meaningful, semantically searchable pieces, once in full and then incrementally forever after.

3.1 First time a repo is connected — full index

A repos table tracks one row per connected repository, with a `last_indexed_sha` column. When this value is `null`, the system has never indexed the repo, so it performs a full pass: shallow-clones the repository (depth 1, no full history needed), walks every source file, parses each with tree-sitter, embeds every extracted chunk, and persists all of it. Only after this succeeds does it write the current commit SHA into `last_indexed_sha` — if anything fails partway through, the value is left as it was, so the next attempt retries a full index instead of assuming partial work is complete.

3.2 Every push after that — incremental index

`last_indexed_sha` is the system's memory of exactly which commit its index currently reflects. On a later push, GitHub's compare API (`GET /repos/{owner}/{repo}/compare/{last_indexed_sha}...{new_sha}`) is asked directly which files changed between that remembered commit and the new one — no local diffing, no re-cloning. Each file comes back tagged added, modified, removed, or renamed, and only those files are fetched and reprocessed.

3.3 Parsing with tree-sitter

Tree-sitter parses one file at a time into a typed syntax tree where every node carries exact start/end byte offsets. A query (written once per language, in tree-sitter's own query syntax) captures the nodes worth indexing — method and class declarations — and for each match, the matched node's byte range is sliced directly out of the original source text. That slice, a whole function including its signature and body, becomes one chunk. This is why chunking is symbol-level rather than fixed-size text windows: a function is a coherent unit of meaning, and cutting one in half destroys that.

```
(method_declaration
  name: (identifier) @method.name) @method.definition
```

Example tree-sitter query capturing every method declaration and its name.

3.4 Reconciling changed files against existing chunks

For a modified file, tree-sitter re-parses the whole file (fast, local, no API cost) and produces chunks for every method in it again — including ones that didn't change. Each new chunk's content hash (a SHA-256 of its exact text) is compared against what's already stored for that symbol. Identical hash: skip entirely, no re-embedding. Different hash, or a new symbol: send it to the embedding model. A symbol that existed before but is missing from the new parse was deleted from the file, and its stored chunk is removed. Deleted files skip parsing entirely (their chunks are just deleted by file path); renamed files only get a metadata update, since their content, and therefore their embedding, hasn't changed.

3.5 Turning a chunk into a vector

The chunk's raw text is sent to the embedding model, which returns a fixed-length vector of floating point numbers (commonly 768 or 1536 dimensions). The key property: code that does something semantically similar produces a vector that lands numerically close to this one, even when variable names, comments, and exact syntax are completely different. This is what later lets an agent find "an existing validation method with a similar null-check-then-throw shape" even when that method shares zero words with the new code being reviewed.

3.6 Multiple files, and parallelism

Tree-sitter itself only ever parses one file at a time; there is no multi-file API. The system walks the repository, filters out generated/vendored directories and unsupported extensions, then parses each remaining file independently — since files don't depend on each other during parsing, this step parallelizes cleanly across threads. Parsing (per file) is kept separate from embedding (batched across many files at once): all chunks from all files in a run are collected first, then sent to the embedding model in batches of, e.g., 32 at a time, rather than one HTTP call per chunk. A single malformed or unsupported file is caught and skipped so it can't abort indexing the rest of the repository.

3.7 How retrieval uses the index at review time

When a PR is reviewed, the context builder extracts the clean new code from the diff (not raw diff markup), embeds it with the same embedding model used during indexing, and queries pgvector for the nearest existing chunks by cosine distance, applying a distance threshold so irrelevant matches are dropped rather than forced into the prompt. The retrieved chunks are inserted directly into the agent's prompt, side by side with the new code, and the agent's judgment ("this doesn't match the established pattern") comes entirely from comparing those two concrete pieces of text in that single LLM call — not from any broader understanding of the project. Indexing does not give any agent comprehension of the codebase; it only makes fast, relevant retrieval possible for one change at a time.

4. Data Model

The system uses two different kinds of storage for two different lifetimes: Postgres for durable, queryable records, and Redis for short-lived per-review working memory.

4.1 Postgres tables

repos	One row per connected repository. Tracks indexing state via last_indexed_sha.
code_chunks	One row per indexed function/method: text, metadata, and embedding vector.
reviews	One row per PR review produced, with the final synthesized comment.
review_findings	One row per individual agent finding, linked back to a review, with provenance.

```
CREATE TABLE repos (  
  repo_id          BIGINT PRIMARY KEY,  
  full_name        TEXT NOT NULL,  
  last_indexed_sha VARCHAR(40),  
  indexing_status  TEXT DEFAULT 'pending',  
  last_indexed_at  TIMESTAMP  
);  
  
CREATE TABLE code_chunks (  
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  repo_id         BIGINT NOT NULL REFERENCES repos(repo_id),  
  file_path       TEXT NOT NULL,  
  symbol_name     TEXT NOT NULL,  
  symbol_type     TEXT NOT NULL,  
  start_line      INT NOT NULL,  
  end_line        INT NOT NULL,  
  content         TEXT NOT NULL,  
  content_hash    TEXT NOT NULL,  
  embedding       VECTOR(768),  
  updated_at      TIMESTAMP DEFAULT now()  
);  
CREATE INDEX ON code_chunks USING ivfflat (embedding vector_cosine_ops);  
  
CREATE TABLE reviews (  
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  repo_id         BIGINT NOT NULL REFERENCES repos(repo_id),  
  pr_number       INT NOT NULL,  
  head_sha        VARCHAR(40) NOT NULL,  
  synthesized_comment TEXT NOT NULL,  
  agents_run      TEXT[],  
  created_at      TIMESTAMP DEFAULT now()  
);  
  
CREATE TABLE review_findings (  
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  review_id       UUID NOT NULL REFERENCES reviews(id),  
  agent_name      TEXT NOT NULL,  
  finding_type    TEXT NOT NULL,  
  description     TEXT NOT NULL,  
  severity        TEXT NOT NULL,  
  cited_chunks    TEXT[],  
  was_included_in_final_comment BOOLEAN  
);
```

4.2 Redis blackboard (not a table — per-PR JSON state)

Redis holds no fixed schema, just one JSON blob per PR under a key like `pr:{repo_id}:{pr_number}`, containing the context bundle and each agent's findings as they complete. It is deliberately short-lived (TTL'd): once a review finishes, the durable record moves into the `reviews / review_findings` Postgres tables, and the Redis entry is discarded.

5. Repository / File Structure

Structured as a multi-module monorepo, since the services share event schemas and data models that would otherwise drift out of sync across separate repositories.

```
code-review-platform/
|-- pom.xml                # parent POM, shared dependency versions
|-- docker-compose.yml     # Kafka, Redis, Postgres+pgvector, all 6 services
|-- docker-compose.override.yml # local dev overrides
|
|-- shared-kernel/         # depended on by every service
|   |-- src/main/java/.../shared/
|   |   |-- events/       # Kafka event payload records (the service contract)
|   |   |   |-- PrSubmittedEvent.java
|   |   |   |-- RepoIndexRequestedEvent.java
|   |   |   |-- AgentCompletedEvent.java
|   |   |   |-- ReviewSynthesizedEvent.java
|   |   |-- model/
|   |   |   |-- CodeChunk.java
|   |   |   |-- Finding.java
|   |   |   |-- ContextBundle.java
|   |   |-- kafka/
|   |   |   |-- Topics.java # topic name constants
|   |
|   |-- gateway-service/
|   |   |-- src/main/java/.../gateway/
|   |   |   |-- controller/GithubWebhookController.java # signature verification, entry point
|   |   |   |-- service/WebhookDispatcher.java
|   |   |   |-- config/KafkaProducerConfig.java
|   |
|   |-- indexer-service/
|   |   |-- src/main/java/.../indexer/
|   |   |   |-- kafka/IndexRequestListener.java
|   |   |   |-- git/RepoFetcher.java # JGit shallow clone / compare API calls
|   |   |   |-- parsing/
|   |   |   |   |-- TreeSitterParserFactory.java
|   |   |   |   |-- ChunkExtractor.java
|   |   |   |   |-- queries/
|   |   |   |   |   |-- java-symbols.scm
|   |   |   |   |   |-- typescript-symbols.scm
|   |   |   |-- embedding/EmbeddingClient.java
|   |   |   |-- persistence/
|   |   |   |   |-- CodeChunkRepository.java
|   |   |   |   |-- RepoRepository.java
|   |   |   |-- service/IndexingOrchestrationService.java
|   |
|   |-- embedding-sidecar/ # Python, deliberately isolated from the Java build
|   |   |-- requirements.txt # sentence-transformers, fastapi, uvicorn
|   |   |-- app/
|   |   |   |-- main.py # POST /embed
|   |   |   |-- model_loader.py # loads the model once at startup
|   |
|   |-- orchestrator-service/
|   |   |-- src/main/java/.../orchestrator/
|   |   |   |-- kafka/PrSubmittedListener.java
|   |   |   |-- service/TaskDecompositionService.java
|   |   |   |-- redis/BlackboardInitializer.java
|   |
|   |-- context-builder-service/
|   |   |-- src/main/java/.../contextbuilder/
|   |   |   |-- kafka/DispatchListener.java
|   |   |   |-- ast/DiffAstAnalyzer.java
|   |   |   |-- retrieval/SemanticSearchClient.java
|   |   |   |-- blackboard/ContextBundleWriter.java
|   |
|   |-- agent-runtime-service/ # one codebase, deployed as N configurations
|   |   |-- src/main/java/.../agent/
|   |   |   |-- config/AgentProfile.java # which topic / tools this instance uses
```

```

|         |-- tools/
|         |   |-- CodeAnalyzerTool.java
|         |   |-- DependencyManagerTool.java
|         |   |-- StyleGuideRetrievalTool.java
|         |   |-- CoverageGapTool.java
|         |-- kafka/AgentTaskListener.java
|         |-- service/AgentReasoningService.java
|
|-- aggregator-service/
    |-- src/main/java/.../aggregator/
        |-- kafka/AgentCompletedListener.java
        |-- service/
            |-- SynthesisService.java
            |-- ConflictResolutionService.java
        |-- github/GithubCommentPoster.java

```

Why this shape

- **shared-kernel** prevents event-schema drift: if an event gains a field, every dependent service gets a compile error until it's handled, instead of a runtime surprise from loosely-typed JSON.
- **embedding-sidecar** is the one deliberately non-Java service, kept isolated so a Python environment never touches the Maven build.
- **agent-runtime-service** is one codebase, not five separate repositories — deployed multiple times with different configuration (which Kafka topic to consume, which tools to register) rather than maintaining near-duplicate services.
- Each service mirrors the same internal shape (a folder for Kafka handling, a folder for the core service logic, plus whatever is domain-specific to it), so understanding one service's layout transfers directly to the rest.

6. Deployment Shape

The system is built as microservices rather than a monolith — a deliberate choice, not a default. The specialized agents have different resource profiles (LLM-bound vs. CPU-bound), the components are already decoupled through Kafka and the blackboard (no request/response coupling between them to begin with), and the indexer runs on an entirely different trigger and cadence than the PR review path, so it could never share a deployment unit with it anyway.

Service	Deployed as
Gateway	1 instance
Orchestrator	1 instance
Indexer	1 instance, separate trigger/cadence from the review path
Context builder	1+ instances, scales with PR volume
Agent runtime	N configurations (one per agent type), independently scalable per type
Aggregator	1 instance

Deployment path: Docker Compose for local development (one file, all six services plus Kafka, Redis, and Postgres); a Helm chart on Minikube is a reasonable stretch goal once the core system is working, but is not required for the project to be complete or resume-credible.

A note on data exposure

Every LLM call and every hosted-embedding call sends the text in its prompt to a third-party API; static analysis, security scanning, and AST parsing stay entirely local. Two independent configuration flags are worth building in from the start: `embedding.provider` (local vs. hosted) and `llm.provider` (local vs. hosted) — since embeddings can be run fully locally at near-zero quality cost, while the LLM reasoning calls trade real capability for privacy. Documenting which components touch a third-party API, and under what data-retention terms, is itself part of the deliverable for a project like this.